

Imagine a university maintains its student records in a system where the records are stored in **sorted order of roll numbers**.

The system behaves like an **infinite sorted array**:

- The array contains integers in **non-decreasing order**.
- You can access any valid index.
- You **do not know the size of the array** beforehand.
- There is no direct way to determine the last index.
- Your task is to find the position of a given **target** roll number.
- Array can contain duplicates as well as negative elements.

For example, the array may look like:

arr = [2, 5, 8, 12, 16, 21, 27, 35, 42, 50, 63, 78, 91, 105, ...]

Notice that we don't know where the array ends.

Input

You are given:

- An infinite sorted array **arr**.
- An integer **target** representing the roll number you want to find.

Output

Return the **index** at which **target** occurs.

If the target does not exist in the array, return **-1**.

Example 1

arr = [2, 5, 8, 12, 16, 21, 27, 35, 42, 50, ...]
target = 35

Output:

7

Example 2

arr = [2, 5, 8, 12, 16, 21, 27, 35, 42, 50, ...]
target = 30

Output:-1

Solution:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int binarySearch(vector<int>& arr, int start, int end, int target) {
```

```
    while (start <= end) {
```

```
        int mid = start + (end - start) / 2;
```

```
        if (arr[mid] == target) {
```

```
            return mid;
```

```
        }
```

```
        else if (arr[mid] < target) {
```

```
            start = mid + 1;
```

```
        }
```

```
        else {
```

```
            end = mid - 1;
```

```
        }
```

```
    }
```

```
    return -1;
```

```
}
```

```
int searchInfiniteArray(vector<int>& arr, int target) {
```

```
// If the first element itself is the target
if (arr[0] == target) {
    return 0;
}

int start = 0;
int end = 1;

// Find a range where target can exist
while (arr[end] < target) {
    start = end;
    end = end * 2;
}

// Binary search in the identified range
return binarySearch(arr, start, end, target);
}

int main() {

    vector<int> arr = {
        2, 5, 8, 12, 16, 21, 27, 35,
        42, 50, 63, 78, 91, 105
    };

    int target = 35;
```

```
cout << searchInfiniteArray(arr, target);

return 0;

}
```

You are standing outside a closed room. Inside the room, there are 3 light bulbs.

Outside the room, there are 3 switches. Each switch controls exactly one bulb, but you don't know which switch controls which bulb.

You can turn the switches ON or OFF as many times as you want while you are outside.

However, there is one rule:

- * You can enter the room only once.
- * After entering the room, you cannot touch the switches.
- * You can only look at the bulbs and check whether they are ON, OFF, or warm.

Question:

How can you find out which switch is connected to which bulb?

Solution:

- Let the three bulbs be A, B, and C, each controlled by one of the switches (switches 1, 2, and 3).
- Turn on switch 1 and leave it on for 5–10 minutes so that its bulb becomes warm, then turn it off.
- Turn on switch 2 and leave it on.
- Keep switch 3 off at all times.
- Now enter the room and observe the bulbs.
- The ON bulb is controlled by switch 2.

- The bulb that is OFF but warm is controlled by switch 1.
- The bulb that is OFF and cold is controlled by switch 3.

A delivery company has n warehouses. Each warehouse has a certain number of packages that need to be loaded onto trucks.

The number of packages in each warehouse is given in the array `boxes`.

A loading team has only h hours to finish loading all the packages.

In one hour, the team can load at most k packages from **one warehouse**.

If a warehouse has fewer than k packages remaining, the team loads all of them and that hour ends.

The team wants to choose the **slowest possible loading speed** while still finishing all the packages within h hours.

Return the minimum value of k .

Example

`boxes = [5, 9, 12, 7]`
`h = 8`

Output: 5

`boxes = [10, 10, 10]` `h = 5`

At a speed of 5 packages/hour:

5 → 1 hour
 9 → 2 hours
 12 → 3 hours
 7 → 2 hours

Total = 8 hours

Therefore, the answer is 5.

Solution:

```
#include <iostream>
```

```

#include <vector>
using namespace std;

bool isPossible(vector<int>& boxes, int mid, int h) {
    long long hours = 0;

    for (int i=0;i<boxes.size();i++) {
        hours+=(boxes[i]/mid);
        if(boxes[i]%mid!=0){
            hours+=1;
        }
    }
    if (hours > h)
        return false;

    return true;
}

int minimumSpeed(vector<int>& boxes, int h) {

    int low = 1;
    int high = 0;

    // Maximum speed needed
    for (int x : boxes) {
        high = max(high, x);
    }

    int answer = high;

    while (low <= high) {

        int mid = low + (high - low) / 2;

        if (isPossible(boxes, mid, h)) {
            // mid works, try a slower speed
            answer = mid;
            high = mid - 1;
        }
        else {
            // mid is too slow
            low = mid + 1;
        }
    }

    return answer;
}

```

```
int main() {  
  
    vector<int> boxes = {10, 10, 10};  
    int h = 5;  
  
    cout << minimumSpeed(boxes, h);  
  
    return 0;  
}
```